

4

Exact Inference

We now turn our attention to the problem of *inference* in graphical models (Figure 4.1). Generally speaking, inference is the process of drawing conclusions from available evidence. Inference can also be framed as a “reasoned change in view” (Harman, 1986) or “reasoning with beliefs” (Boghossian, 2014). Through this lens, inference can be a dynamic process where beliefs are iteratively challenged and updated as new evidence is introduced.

Given a probabilistic model (such as a Bayesian network or MRF), we perform inference to answer useful questions about the world. Our model can help us answer questions like:

- *What is the posterior distribution over a variable of interest?*
- *What is the most probable explanation for a given hypothesis?*
- *How certain are we that this explanation holds?*

In this chapter, we present means of answering such questions. We will mainly consider the problem of computing posterior probabilities of the form $p(\mathbf{y} \mid \mathbf{x})$ for *query variables* $\mathbf{y}$ given *observed evidence* $\mathbf{x} = (x_1, \dots, x_n)$ (Zhang and Poole, 1996). Formally, we will focus on two types of queries:

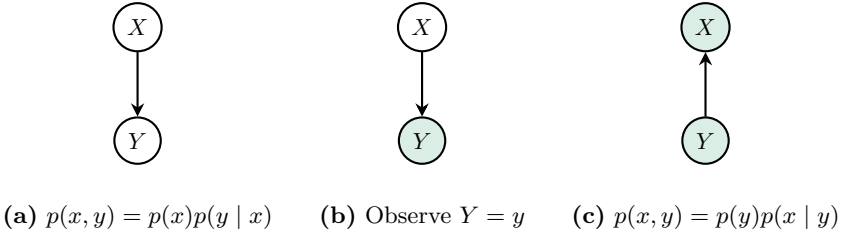


Figure 4.1: A simple case of inference over a graphical representation of Bayes' rule (figure and caption adapted from Bishop 2006, Chapter 8.4). Let $p(x, y)$ be a joint distribution over variables X and Y . We can factorize this joint as $p(x)p(y | x)$ by applying the chain rule (Definition 2.28), as graphically represented by (a). Next, we observe the value taken by Y (denoted by shading) and treat $p(x)$ as a prior over the latent variable X (b). We now wish to infer the posterior distribution $p(x | y)$. By the sum and product rules, $p(y) = \sum_{x'} p(y | x')p(x')$. By Bayes' rule (Definition 2.29), $p(x | y) = p(y | x)p(x)/p(y)$. Thus, we can rearrange the joint factorization as $p(y)p(x | y)$, as represented by (c).

1. *Marginal inference.* What is the probability of a given variable in our model after we sum everything else out, e.g.,

$$p(y = 1) = \sum_{x_1} \sum_{x_2} \cdots \sum_{x_n} p(y = 1, x_1, x_2, \dots, x_n).$$

2. *Maximum a posteriori (MAP) inference.* What is the most likely assignment to the variables in the model, e.g.,

$$\hat{\mathbf{x}} = \operatorname{argmax}_{x_1, \dots, x_n} p(y = 1, x_1, \dots, x_n).$$

In this chapter, we place our initial focus on *exact* probabilistic inference in graphical models. This is a challenging task that is NP-hard in the general case (Cooper, 1990). Nevertheless, tractable solutions can be obtained for certain kinds of problems. As illustrated throughout this chapter, the tractability of an inference problem depends heavily on the structure of the graph that describes the probability of interest.

To begin, we will introduce three fundamental and inter-related algorithms for exact inference: variable elimination (Section 4.1), belief propagation for tree-structured graphs (Section 4.2), and the junction tree algorithm (Section 4.3). We will then discuss the computational complexity of MAP versus marginal inference and present a couple of

algorithms for efficient MAP inference (Section 4.4). In Chapter 5, we will continue our discussion in the context of *approximate inference* for the setting where exact inference is intractable.

4.1 Variable Elimination

We first introduce *variable elimination*, a fundamental exact inference method for answering queries to MRFs and Bayesian networks (Zhang and Poole, 1994). Variable elimination is a general approach that can be used for both marginal and MAP inference. It efficiently computes quantities of interest by serially *eliminating* variables that are irrelevant to the query. Given a model over variables $\mathbf{X}$, with joint distribution $p(\mathbf{x})$, variable elimination can be used to answer queries such as conditional probabilities — for example, computing a distribution of the form

$$p(\mathbf{x}_q \mid \mathbf{x}_e),$$

where $\mathbf{X}_q$ is a set of query variables, $\mathbf{X}_e$ is a (potentially empty) set of evidence variables, and the remaining variables in $\mathbf{X}$ are marginalized out. Note that for MAP inference, this query will take a different form. At a high-level, variable elimination exploits two convenient observations (Koller and Friedman, 2009):

1. By the factorization of Bayesian networks (Definition 3.2), factors often depend on a relatively small subset of variables in the joint. We can reduce the number of parameters in our model by leveraging this fact (see Section 3.1.1).
2. With dynamic programming, we can compute subexpressions in the joint and memoize to prevent exponential increases in computation.

This procedure repeatedly performs two factor operations: *product* and *marginalization*. Conveniently, we can interchange product and sum operations to reduce the number of computations. For example, consider three factors: ϕ_x with scope $\mathbf{X}$, and ϕ_y with scope $\mathbf{Y}$, and ϕ_{yz} with scope $\mathbf{Y} \cup \mathbf{Z}$. Because $\mathbf{Y}$ is in scope for ϕ_y and ϕ_{yz} only, we can interchange

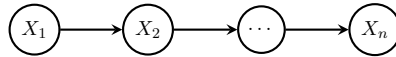


Figure 4.2: A chain-structured Bayesian network with n variables.

sum and product as follows:

$$\sum_{\mathbf{y}} \phi_x \phi_y \phi_{yz} = \phi_x \sum_{\mathbf{y}} \phi_y \phi_{yz}.$$

This simple rearrangement, whereby we *push* the summation deeper into the product, is the crux of variable elimination.

For ease of exposition, we will assume for the remainder of this chapter that all random variables are discrete variables taking k possible values each. While the principles behind variable elimination also extend to many continuous distributions (e.g., Gaussians), we will not discuss these extensions here.

We present the full variable elimination procedure in Algorithm 1. To lay the groundwork, we first describe the special case of marginal inference in chain-structured graphs. Building on this, we then present the general case, known as the *sum-product variable elimination algorithm*, and describe how it can be used for both marginal and MAP inference queries, as well as queries that condition on observed evidence.

4.1.1 Marginal Inference in a Chain-Structured Graph

We will first use a simple chain-structured graph as an illustrative example for several useful concepts.

Suppose for simplicity that we are given a Bayesian network that resembles Figure 4.2. This network represents a joint probability distribution of the form

$$p(x_1, \dots, x_n) = p(x_1) \prod_{i=2}^n p(x_i \mid x_{i-1}).$$

Assume that we are interested in computing the marginal probability $p(x_n)$. The naive way of calculating this is to sum the probability over all k^{n-1} assignments to $x_1, \dots, x_{n-1}$:

$$p(x_n) = \sum_{x_1} \cdots \sum_{x_{n-1}} p(x_1, \dots, x_n).$$

However, we can do much better by leveraging the factorization of our joint probability distribution. This factorization allows us to rewrite the sum in a way that pushes certain variables deeper into the product.

$$\begin{aligned} p(x_n) &= \sum_{x_1} \cdots \sum_{x_{n-1}} p(x_1) \prod_{i=2}^n p(x_i \mid x_{i-1}) \\ &= \sum_{x_{n-1}} p(x_n \mid x_{n-1}) \sum_{x_{n-2}} p(x_{n-1} \mid x_{n-2}) \cdots \sum_{x_1} p(x_2 \mid x_1) p(x_1). \end{aligned}$$

We sum the inner terms first, starting from x_1 and ending with x_{n-1} . Concretely, we start by computing an intermediary factor

$$\tau(x_2) = \sum_{x_1} p(x_2 \mid x_1) p(x_1)$$

by summing out x_1 . This takes $O(k^2)$ time because we must sum over x_1 for each assignment to x_2 . The resulting factor $\tau(x_2)$ can be thought of as a table of k values (though not necessarily probabilities), with one entry for each assignment to x_2 (just as factor $p(x_1)$ can be represented as a table). We can then rewrite the marginal probability using τ as

$$p(x_n) = \sum_{x_{n-1}} p(x_n \mid x_{n-1}) \sum_{x_{n-2}} p(x_{n-1} \mid x_{n-2}) \cdots \sum_{x_2} p(x_3 \mid x_2) \tau(x_2).$$

Note that this has the same form as the initial expression, except that we are summing over one fewer variable. Thus, we are sequentially *eliminating* one variable at a time, which gives the algorithm its name.

We can therefore compute another factor

$$\tau(x_3) = \sum_{x_2} p(x_3 \mid x_2) \tau(x_2),$$

and repeat the process until we are only left with x_n , which yields a table of probabilities representing $p(x_n)$. Since each step takes $O(k^2)$ time, and we perform $O(n)$ steps, inference now takes $O(nk^2)$ time, which is much better than our naive $O(k^n)$ solution.

4.1.2 The Sum-Product Variable Elimination Algorithm

While the previous example illustrated marginal inference in a chain-structured graph, we now present variable elimination as a general

procedure applicable to arbitrary graphical models with factorized representations — including both Bayesian networks and MRFs. Later, we show how, in addition to marginal inference, this procedure can be used for MAP inference and computing conditional distributions.

We first focus here on the *sum-product formulation* of variable elimination. This procedure repeatedly performs two factor operations: *product* and *marginalization*. Given two factors ϕ_1 and ϕ_2 , the factor product operation simply defines the product $\phi_3 := \phi_1 \times \phi_2$ as

$$\phi_3(\mathbf{x}) = \phi_1(\mathbf{x}^{(1)}) \times \phi_2(\mathbf{x}^{(2)}).$$

The scope of ϕ_3 is defined as the union of the variables in the scopes of ϕ_1 and ϕ_2 . We use $\mathbf{x}^{(i)}$ to denote an assignment to the variables in the scope of ϕ_i defined by the restriction of $\mathbf{x}$ to that scope. For example, we define $\phi_3(a, b, c) := \phi_1(a, b) \times \phi_2(b, c)$.

Next, the marginalization operation “locally” eliminates a set of variables from a factor. If we have a factor $\phi(\mathbf{x}, \mathbf{y})$ over two sets of variables $\mathbf{X}$ and $\mathbf{Y}$, marginalizing $\mathbf{Y}$ produces a new factor

$$\tau(\mathbf{x}) = \sum_{\mathbf{y}} \phi(\mathbf{x}, \mathbf{y}),$$

where the sum is over all joint assignments to the set of variables $\mathbf{Y}$. We use τ to refer to the marginalized factor. It is important to understand that this factor does not necessarily correspond to a probability distribution, even if ϕ was a conditional probability distribution.

Given the definitions of the product and marginalization operations over factors, the sum-product variable elimination algorithm proceeds as follows. Suppose we are given a graphical model $\mathcal{G} = (\mathbf{V}, \mathbf{E})$ with an associated set of d factors $\Phi := \{\phi_i\}_{i=1}^d$, and we wish to compute the marginal distribution over a subset of query variables $\mathbf{X}_q \subseteq \mathbf{V}$, i.e., we aim to compute $p(\mathbf{x}_q)$. To do this, we must eliminate the remaining variables $\mathbf{X}_m := \mathbf{V} \setminus \mathbf{X}_q$ by summing them out. We begin by selecting an *elimination ordering* over the variables in $\mathbf{X}_m$ (see Section 4.1.4); for instance, in a Bayesian network, a topological ordering is one possible choice. Then, for each variable in the ordering, we identify the subset of factors in Φ that mention the variable, and on these we perform the product factor operation and then the marginalization factor operation

Algorithm 1 *Sum-Product Variable Elimination*, adapted from Zhang and Poole (1996) and Koller and Friedman (2009)

Input:

- $\Phi := \{\phi_i\}_{i=1}^d$: Set of factors implied by graph $\mathcal{G} = (\mathbf{V}, \mathbf{E})$
- $\mathbf{X}_q \in \mathbf{V}$: Set of query variables
- $\mathcal{O}$: Elimination ordering for $\mathbf{V} \setminus \mathbf{X}_q$

Output: $\phi^* = p(\mathbf{x}_q)$

- 1: **for** $X \in \mathcal{O}$, following the elimination ordering **do**
 - 2: Remove variable X from $\mathcal{O}$
 - 3: Remove from Φ all $\{\phi_j\}_{j=1}^k$ where $X \in \text{Scope}[\phi_j]$
 - 4: Add new factor $\sum_x \prod_{j=1}^k \phi_j$ to Φ
 - 5: $\phi^* \leftarrow \prod_{i=1}^d \phi_i$
 - 6: **return** ϕ^*
-

(which sums out the given variable). The result is a new factor that replaces this subset of factors in Φ , and the process continues. This sequentially reduces the number of factors and eliminates variables one by one. Conceptually, the algorithm pushes summations as far inside the product of factors as possible, as illustrated in the chain-structured example. At the end of the procedure, a single factor remains, defined over the query variables $\mathbf{X}_q$, which represents the unnormalized marginal distribution $p(\mathbf{x}_q)$. The full pseudocode for the sum-product variable elimination algorithm is provided in Algorithm 1.

Two Illustrative Examples

To make Algorithm 1 concrete, we revisit the chain-structured example from Figure 4.2 to illustrate how these steps play out. In this case, the chosen ordering was simply the topological ordering $x_1, x_2, \dots, x_{n-1}$. Starting with x_1 , we collected all the factors involving x_1 , which were $p(x_1)$ and $p(x_2 \mid x_1)$. We then used them to construct a new factor

$$\tau(x_2) = \sum_{x_1} p(x_2 \mid x_1)p(x_1).$$

This can be seen as the results of Steps 1 and 2 of variable elimination in Algorithm 1: first we form a large factor $p(x_2, x_1) = p(x_2 \mid x_1)p(x_1)$,

then we eliminate x_1 from that factor to produce τ . Next, we repeat the same procedure for x_2 , except that the factors are now $p(x_3 | x_2), \tau(x_2)$.

As a second illustrative example of the variable elimination algorithm, we walk through a slightly more complex application of Algorithm 1. Recall the graphical model of student performance that we introduced earlier (Figure 3.1). The probability specified by the model is of the form

$$p(l, g, i, d, s) = p(l | g)p(s | i)p(i)p(g | i, d)p(d).$$

Let's suppose that we are computing $p(l)$ and are eliminating variables in their topological ordering in the graph. First, we eliminate d , which corresponds to creating a new factor

$$\tau_1(g, i) = \sum_d p(g | i, d)p(d).$$

Next, we eliminate i to produce a factor

$$\tau_2(g, s) = \sum_i \tau_1(g, i)p(i)p(s | i).$$

Then, we eliminate s yielding $\tau_3(g) = \sum_s \tau_2(g, s)$, and so on. Note that these operations are equivalent to summing out the factored probability distribution as follows:

$$p(l) = \sum_g p(l | g) \sum_s \sum_i p(s | i)p(i) \sum_d p(g | i, d)p(d).$$

This example requires computing at most k^3 operations per step: each factor is at most over 2 variables, and one variable is summed out at each step (the dimensionality k in this example is either 2 or 3).

Computational Complexity

Though variable elimination is often faster than the naive approach of inference by enumeration, it is still NP-hard. In short, the computational complexity of variable elimination is exponential in the size of the largest factor. However, it is very important to understand that the running time of variable elimination depends heavily on the structure of the graph. More concretely, the running time depends on the following structural properties of the Bayesian network or MRF $\mathcal{G}$.

- *Total variables in $\mathcal{G}$.* Let $n = |\mathbf{V}|$ denote the number of variables in the system.
- *Treewidth.* Treewidth w , which measures how close the graph is to a tree (Definition 2.43). In this setting, the treewidth can be shown to be equivalent to the maximum size of any factor τ formed during the variable elimination algorithm.
- *Domain size of variables in $\mathbf{V}$.* Let k_i denote the size of the domain (i.e., the number of possible values) for each $V_i \in \mathbf{V}$. For ease of exposition, we can assume that all variables have k states.

Thus, we can express the time complexity of variable elimination as $O(nk^{w+1})$. As $w = 1$ in trees (including chains), the computational complexity of variable elimination on tree-structured graphs is linear in n : $O(nk^2)$. For graphs with large treewidth, variable elimination can become intractable for large n .

4.1.3 Introducing Evidence

Above we have described the variable elimination algorithm for the case of marginal inference. A closely related and equally important problem is the computation of conditional probabilities of the form

$$p(\mathbf{x}_q \mid \mathbf{x}_e = \mathbf{e}) = \frac{p(\mathbf{x}_q, \mathbf{x}_e = \mathbf{e})}{p(\mathbf{x}_e = \mathbf{e})},$$

where $p(\mathbf{x}_q, \mathbf{x}_e, \mathbf{x}_m)$ is a probability distribution over *query* variables $\mathbf{X}_q$, *observed evidence* variables $\mathbf{X}_e$, and *unobserved* variables $\mathbf{X}_m$.

We can compute this probability by performing variable elimination once on $p(\mathbf{x}_q, \mathbf{x}_e = \mathbf{e})$ and then once more on $p(\mathbf{x}_e = \mathbf{e})$. To compute $p(\mathbf{x}_q, \mathbf{x}_e = \mathbf{e})$, we simply take every factor $\phi(\mathbf{x}'_m, \mathbf{x}'_q, \mathbf{x}'_e)$ which has scope over variables $\mathbf{X}'_e \subseteq \mathbf{X}_e$, and we set their values to $\mathbf{e}$. Then we perform standard variable elimination to obtain a factor over $\mathbf{X}_q$ only.

4.1.4 Elimination Orderings

The variable elimination algorithm requires an *ordering* over the variables, according to which variables will be eliminated. In our chain

example, we simply took the ordering implied by the DAG. However, it is important to note that the time complexity of variable elimination is sensitive to the order in which variables are eliminated. Further, it is NP-hard to find the optimal ordering.

In practice, we can resort to the following heuristics to obtain an ordering for variable elimination. These methods often result in reasonably good performance in many interesting settings (Koller and Friedman, 2009).

- *Min-neighbors*, where we choose the next variable with the fewest dependent variables.
- *Min-weight*, where we choose variables to minimize the product of the cardinalities of dependent variables.
- *Min-fill*, where we choose vertices to minimize the size of the factor that will be added to the graph.

4.1.5 Generalizing Factor Operations

We have so far described the sum-product formulation of variable elimination for marginal (and conditional) inference, involving the *product* and *marginalization* factor operators. We now generalize these operations, enabling alternative inference procedures with the variable elimination algorithm, including MAP inference.

The premise of variable elimination rests on the properties of *commutative semirings*: a set $\mathbf{S}$ and two binary operations that are associative, commutative, and satisfy identity laws. Commutative semirings satisfy the distributive law, whereby

$$(x \times y) + (x \times z) = x \times (y + z) \quad (4.1)$$

for all $x, y, z \in \mathbf{S}$. Note that the distributive law reduced the number of computations in Equation 4.1 from three (on the lefthand side) to two (on the righthand side). See Golan (2013) for a formal discussion of commutative semirings.

The associative, commutative, and distributive properties of these binary operations allow us to perform more efficient inference. When the

	DOMAIN	+	×
<i>Sum-product</i>	$[0, \infty)$	$(+, 0)$	$(\times, 1)$
<i>Max-product</i>	$[0, \infty)$	$(\max, 0)$	$(\times, 1)$
<i>Min-sum</i>	$(-\infty, \infty]$	$(\min, \infty)$	$(+, 0)$
<i>Boolean satisfiability</i>	$\{T, F\}$	$(\vee, F)$	$(\wedge, T)$

Table 4.1: Examples of common commutative semirings. Table from Murphy (2012).

binary operations of a commutative semiring are taken to be addition and multiplication, we obtain the sum-product formulation of variable elimination:

$$p(\mathbf{x}_q \mid \mathbf{x}_e) \propto \sum_{\mathbf{x}} \prod_{c \in \mathbf{C}} \phi_c(\mathbf{x}_c),$$

where $\mathbf{X}_q$ is a set of query variables, $\mathbf{C}$ is a set of cliques c , and $\mathbf{X}_e$ is a set of evidence variables whose values are fixed rather than marginalized over (Murphy, 2012). Notably, replacing summation with the *max* operation in variable elimination yields a version of the algorithm for MAP inference. See Table 4.1 for additional commutative semirings.

4.1.6 Limiting Extraneous Computation

We have seen how the variable elimination algorithm can answer marginal queries of the form $p(\mathbf{x}_q \mid \mathbf{x}_e = \mathbf{e})$ for both directed and undirected networks. However, this algorithm has an important shortcoming: if we want to ask the model for another query (e.g., $p(\mathbf{x}'_q \mid \mathbf{x}'_e = \mathbf{e}')$), we need to restart the algorithm from scratch. This is very wasteful and computationally burdensome.

Fortunately, it turns out that this problem is also easily avoidable. When computing marginals, variable elimination produces many intermediate factors τ as a side-product of the main computation. These factors turn out to be the same as the ones that we need to answer other marginal queries. By caching them after a first run of variable elimination, we can easily answer new marginal queries at essentially no additional cost.